

Francisco Javier Díaz Martínez

Título del Proyecto: OPTIBASE.

Desarrollo de una Plataforma Web Integral para la Gestión Clínica y Comercial de una Óptica.

1. Resumen, Alcance y Límites del MVP

El proyecto "Optibase" es una plataforma web para la digitalización de la gestión de pacientes, el inventario básico y la organización de la agenda de una clínica óptica. Se establecen los siguientes límites:

Qué incluye sí o sí:

- Gestión de citas: Crear, modificar y cancelar citas. Bloqueo automático por sistema de creación de citas que se solapen en fecha y hora.
- Ficha de Cliente: Registro de clientes, asociación de graduaciones visuales (historial simulado cronológico) y CRUD completo.
- Inventario Básico: Inventario de artículos (monturas, gafas de sol, líquidos). CRUD completo por el administrador y bloqueo del sistema para impedir registrar/actualizar artículos con stock negativo.
- Seguridad y Roles (RBAC): Autenticación mínima obligatoria.

Qué NO incluye (Fuera del alcance):

- No se enviarán correos electrónicos ni SMS reales de recordatorio de citas.
- No se generará facturación electrónica legal.
- No se incluirá un módulo de recursos humanos (nóminas, vacaciones de empleados).

- Requisitos No Funcionales (NFR) y Criterios de Aceptación: Para asegurar la calidad, la seguridad y la viabilidad del sistema en un entorno real, se definen los siguientes requisitos no funcionales críticos:

1. Protección de Datos (RGPD) y Cifrado:

- NFR: Las contraseñas de los usuarios no pueden almacenarse en texto plano bajo ninguna circunstancia.
- Criterio de aceptación: Se verificará directamente en la consola de la base de datos que las contraseñas de los usuarios están ofuscadas utilizando el algoritmo de hash BCrypt.

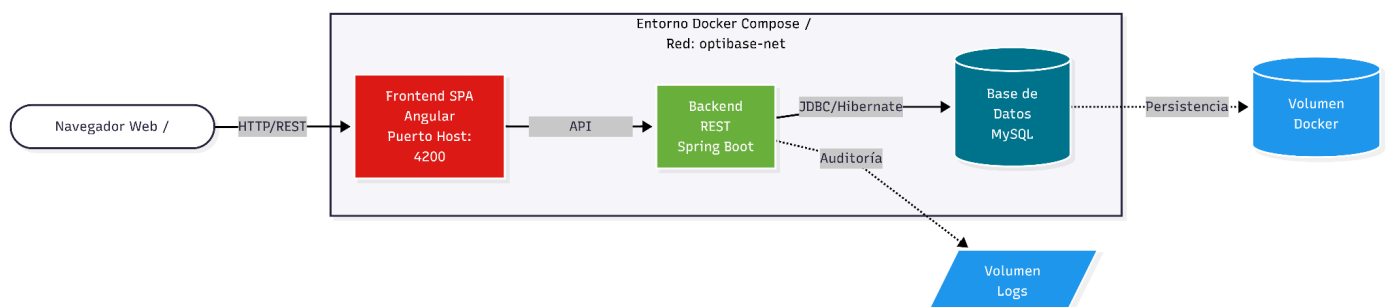
2. Control de Acceso por Recurso (Autorización):

- NFR: Aislamiento estricto de los datos entre clientes a nivel de servidor.
- Criterio de aceptación: Si un Cliente autenticado intenta acceder mediante una petición HTTP (ej. Postman) a un ID de cita o historial clínico que pertenece a otro cliente, el servidor devolverá obligatoriamente un código de error 403 Forbidden.

3. Persistencia y Backups (Resiliencia):

- NFR: Los datos clínicos y el inventario no pueden perderse si el servidor o el contenedor se reinicia de forma inesperada.
- Criterio de aceptación: Se destruirá el contenedor de la base de datos de forma manual (docker rm -f) y, al volver a levantarlo, los datos de pacientes y citas seguirán intactos gracias al correcto mapeo del volumen externo mysql-data.

2. Arquitectura Inicial



Como se ilustra en el diagrama superior, la comunicación (Cliente → Frontend → API → BD) y las responsabilidades se dividen en:

- **Servicio 1: Frontend (Cliente):** Single Page Application (SPA) en Angular. Expone el puerto 4200 al host. *Responsabilidad:* Interfaz de usuario, consumo de API REST y manejo del estado visual.
- **Servicio 2: Backend (API REST):** Desarrollado en Java con Spring Boot. Expone el puerto 8080. *Responsabilidad:* Lógica de negocio, validación de agenda, mapeo JPA/Hibernate y seguridad (JWT). Se montará un volumen `/app/logs` para guardar la auditoría.
- **Servicio 3: Base de Datos:** Motor relacional MySQL. Corre en el puerto interno 3306 (no expuesto al exterior por seguridad). *Responsabilidad:* Persistencia de datos. Utiliza el volumen externo `mysql-data` para evitar la pérdida de información si el contenedor se reinicia.

3. Planificación Semanal, Hitos y Dependencias

Semanas 1-2 | Hito 1: Análisis y Base de Datos

- Tareas: Toma de requisitos, diseño del diagrama E-R, normalización de tablas y creación del script DDL.
- Entregable: Archivo init_db.sql con la estructura completa.
- Verificación: Ejecución del script en un servidor MySQL en blanco sin errores de sintaxis ni de claves foráneas.

Semanas 3-5 | Hito 2: Desarrollo del Backend y Lógica

- Tareas: Inicialización de Spring Boot, mapeo JPA, creación de CRUDs y lógica de solapamiento de citas.
- Entregable: API REST funcional y persistencia automatizada.
- Verificación: Uso de Postman/cURL para realizar operaciones CRUD y comprobar que el endpoint de crear cita bloquea correctamente fechas solapadas devolviendo un error.

Semanas 6-8 | Hito 3: Seguridad (BCrypt/JWT) Y Desarrollo del Frontend

- Tareas: Configuración de filtros Spring Security, ofuscación de contraseñas con BCrypt, proyecto Angular, diseño de vistas (Login, Agenda, Inventario), guardas de rutas (AuthGuard) y consumo de servicios HTTP.
- Entregable: Aplicación cliente SPA navegable y API securizada.
- Verificación: Un usuario puede iniciar sesión desde el navegador obteniendo su token, la base de datos refleja las contraseñas encriptadas, y se puede crear una cita en la interfaz viéndola reflejada inmediatamente en la base de datos.

Semanas 9-10 | Hito 4: Pruebas E2E y Despliegue Docker

- Tareas: Ejecución de pruebas de roles, creación de Dockerfile para frontend y backend, y configuración del orquestador docker-compose.yml.
- Entregable: Proyecto empaquetado y Memoria Final.
- Verificación: El sistema completo (BD, API y Web) se levanta ejecutando un único comando (docker-compose up -d) en una máquina limpia.

5. Entorno de Trabajo y Bitácora Inicial

Estructura de Carpetas del Repositorio:

- optibase-tfg/
- |— backend/ # Proyecto Spring Boot (Maven)
- | |— src/main/java/...
- | |— pom.xml
- |— frontend/ # Proyecto Angular
- | |— src/app/...
- | |— package.json
- |— database/ # Scripts SQL (DDL y DML de prueba)
- | |— init_db.sql
- |— docs/ # Diagramas, mockups y memoria
- |— bitacora.md # Diario de lecciones aprendidas
- |— docker-compose.yml # Orquestador principal
- |— README.md # Instrucciones de despliegue